

Лабораторная работа №7

Дискретное логарифмирование в конечном поле: ρ -метод Полларда

Кюнкриков Даниил Саналович

2026-03-18

Информация

Вводная часть

Содержание исследования

Выполнение работы

Итоговый слайд

1. Информация

Кюнкриков Даниил Саналович

студент уч. группы НПМд-01-24

Российский университет дружбы народов (РУДН)

1132249574@pfur.ru

репозиторий: https://github.com/DanzanK/2025-2026_math-sec/tree/main

2. Вводная часть



Задача дискретного логарифмирования (DLP) — фундаментальная проблема криптографии; на её сложности основаны схемы Диффи–Хеллмана и Эль-Гамала.

Изучение алгоритмов решения DLP важно для оценки стойкости криптосистем и выбора безопасных параметров групп.

Объект исследования: алгоритмы дискретного логарифмирования в конечном поле.

Предмет исследования: задача $a^x \equiv b \pmod{p}$ в группе $\mathbb{F}_p^*$;

ρ -метод Полларда (псевдослучайное блуждание и коллизии);

метод Флойда « черепаха–заяц »;

восстановление x через решение линейного сравнения по модулю r .

Цель: реализовать p -метод Полларда для решения DLP и проверить работу на контрольном примере.

Задачи: описать постановку DLP и роль порядка r элемента a ;

реализовать генерацию последовательности и поиск коллизии (Floyd);

реализовать вычисление x через линейное сравнение и расширенный Евклид;

выполнить тестирование на параметрах (p, a, b, r) .

Язык программирования: **Python**.

Метод: псевдослучайная последовательность по группе и поиск коллизии (Floyd).

Вычисления: модульное умножение, расширенный алгоритм Евклида, решение линейного сравнения по модулю r .

3. Содержание исследования

Найти x , такое что

$$a^x \equiv b \pmod{p}, \quad a, b \in \mathbb{F}_p^*.$$

Если r — порядок элемента a , то x определяется по модулю r .

строим последовательность $c_{i+1} = f(c_i)$ в группе;

поддерживаем представление $c \equiv a^\alpha b^\beta$;

ищем коллизию методом Флойда:

$c \leftarrow f(c)$ (1 шаг),

$d \leftarrow f(f(d))$ (2 шага).

При коллизии $c = d$ получаем:

$$(\beta_1 - \beta_2)x \equiv (\alpha_2 - \alpha_1) \pmod{r}.$$

Обозначим $A = (\beta_1 - \beta_2) \bmod r$, $\Delta = (\alpha_2 - \alpha_1) \bmod r$. Тогда решаем:

$$Ax \equiv \Delta \pmod{r}.$$

Используем расширенный алгоритм Евклида:

$$g = \gcd(A, r),$$

если $g \nmid \Delta \rightarrow$ перезапуск,

иначе находим решение по модулю r/g .

4. Выполнение работы



```
# Расширенный алгоритм Евклида
def egcd(a: int, b: int):
    if b == 0:
        return (a, 1, 0)
    g, y1, x1 = egcd(b, a % b)
    return (g, y1, x1 - (a // b) * y1)
def rho_dlog(p: int, a: int, b: int, r: int):
    # строим псевдослучайную последовательность в группе и ищем коллизию с == d,
    # Случайная инициализация логарифмов (u, v) в диапазоне [0, r-1]
    u = secrets.randbelow(r)
    v = secrets.randbelow(r)
    # Стартовая точка:
    c = (pow(a, u, p) * pow(b, v, p)) % p
    # Для поиска коллизий используем "черепаха-заяц" (Floyd):
    d = c
    alpha1, beta1, alpha2, beta2 = u, v, u, v
    # Простое разбиение множества состояний на 2 части
    half = p // 2

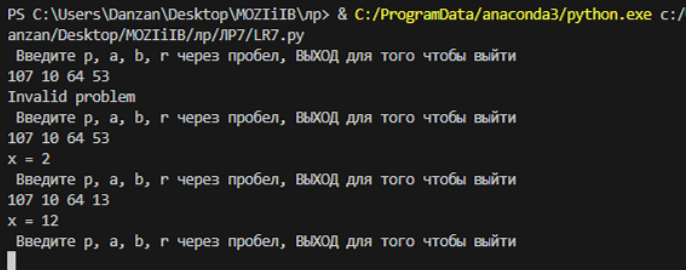
    def step(x: int):
        if x < half:
            return (a * x) % p
        else:
            return (b * x) % p

    # Основной цикл: идём, пока не найдём коллизию c == d
    while True:
        c = step(c)
        alpha1 = (alpha1 + (1 if c < half else 0)) % r
        beta1 = (beta1 + (1 if c >= half else 0)) % r
        d = step(d)
        d = step(d)
        alpha2 = (alpha2 + 2 * (1 if d < half else 0)) % r
        beta2 = (beta2 + 2 * (1 if d >= half else 0)) % r

        if c == d:
            break
    A = (beta1 - beta2) % r
    delta = (alpha2 - alpha1) % r
    # Решаем A*x = delta (mod r)
    # Если g = gcd(A, r) не делит delta — решения нет (Invalid problem)
    g, xcoef, _ = egcd(A, r)
    if delta % g != 0:
        return None
    return (xcoef * (delta // g)) % r
```

Рисунок 1: Программная реализация метода Полларда по разложению чисел на множители

Параметры: $p = 107$, $a = 10$, $b = 64$, $r = 53$.



```
PS C:\Users\Danzan\Desktop\MOZiiB\лр> & C:/ProgramData/anaconda3/python.exe c:/
anzan/Desktop/MOZiiB/лр/ЛР7/LR7.py
Введите p, a, b, r через пробел, ВЫХОД для того чтобы выйти
107 10 64 53
Invalid problem
Введите p, a, b, r через пробел, ВЫХОД для того чтобы выйти
107 10 64 53
x = 2
Введите p, a, b, r через пробел, ВЫХОД для того чтобы выйти
107 10 64 13
x = 12
Введите p, a, b, r через пробел, ВЫХОД для того чтобы выйти
█
```

Рисунок 2: Результат работы программы

Получено корректное значение x (проверка: $10^x \bmod 107 = 64$).

5. Итоговый слайд



Реализован p -метод Полларда для дискретного логарифмирования в $\mathbb{F}_p$.

Реализованы: поиск коллизий (Floyd) и решение линейного сравнения через расширенный Евклид.

На контрольном примере получен корректный результат.